ForecastLabAI — time-safe features against data leakage

Problem

A forecasting model looks great in validation because a feature secretly used the value it is predicting (or the future). Explain how ForecastLabAI prevents that.

Instant solution

```
# Time-safe rolling mean: shift(1) BEFORE rolling, grouped per series
df = df.sort_values(["store_id", "product_id", "date"])
df["rolling_mean_7"] = (
    df.groupby(["store_id", "product_id"])["quantity"]
      .transform(lambda s: s.shift(1).rolling(7).mean())
)
```

- Without `shift(1)`, row *i* uses rows *i–6 … i* (includes the target). With it: *i–7 … i–1*, past only.
- `groupby(store, product)` keeps one series' values from leaking into another.

Feature groups in `featuresets/service.py`: lags, rolling stats (shifted), calendar, exogenous price/stockout, lifecycle, promotion, replenishment, plus imputation.

Interview answer

> *I focused on preventing temporal leakage: chronological training boundaries, lagged features, shifted rolling statistics, and isolated store-product series. A model that sees future information can look excellent in validation, but that performance won't reproduce at inference time.*

Why it works / gotchas

- Leakage = using information at training/evaluation time that would not exist at prediction time.
- The repo's own comment: "CRITICAL: shift(1) BEFORE rolling to exclude current observation."
- Exogenous features (price, promo) must also be **known in the future** at forecast time, or the model can't use them in production.
- Leakage can also come from fold boundaries — see backtesting.

Diagram brief

- Type: timeline (horizontal days t−8 … t).
- Row A "rolling(7) — leaky": window box covers t−6 … t, target t highlighted red inside the window.
- Row B "shift(1).rolling(7) — safe": window covers t−7 … t−1, target t outside, green.
- Side panel: two series (store A/product 1, store B/product 2) with a wall between them labelled "groupby".